

Algorithme à Stratégie Multistart

Edison AIKOUN

August 20, 2026

1 Introduction

Ce document présentera plusieurs algorithmes conçu pour résoudre des problèmes d'optimisation, avec des contraintes de complémentarité (MPCC). D'abord, l'algorithme nommé `tiktak`, qui emploie une stratégie de recherche globale de type multistart pour trouver une solution initiale prometteuse avant une optimisation locale finale.

2 Compréhension Générale du Code Tiktak

Le script peut être divisé en plusieurs parties logiques :

1. Chargement et préparation du modèle d'optimisation avec MadNLP pour faciliter l'initialisation et toucher aux variables.
2. Génération et évaluation d'un grand nombre de points candidats initiaux(methode Sobol)
3. Application d'une stratégie de recherche locale guidée, servant de cœur à l'approche "multistart".
4. Utilisation de la meilleure solution trouvée comme point de départ chaud (warm-start) pour la résolution finale.

2.1 Le Déroulé de l'Algorithme Tiktak

L'algorithme `tiktak` implémente une approche de recherche globale combinée à une optimisation locale. Sa stratégie de multistart est particulièrement intéressante :

1. **Exploration Initiale Large** : L'utilisation de l'échantillonnage de Sobol (`generate_sobol_candidates`) permet de générer un ensemble de points candidats qui couvrent l'espace de recherche de manière efficace, minimisant les corrélations et maximisant la couverture pour un nombre donné de points. Ceci est essentiel pour ne pas manquer de régions potentiellement optimales.
2. **Filtrage des Candidats Prometteurs** : En évaluant tous les candidats Sobol (`f_obj`), le code identifie les points qui donnent les meilleures valeurs pour la fonction objectif. Ces points (`candidat_garde`) forment un ensemble réduit des régions les plus intéressantes de l'espace de recherche.
3. **Recherche Locale Itérative et Guidée (cherche)** : C'est ici que réside la particularité de l'approche "multistart" de `tiktak`. Plutôt que de simplement lancer des recherches locales à partir de chaque `candidat_garde`, la fonction "`cherche`" fait :
 - La boucle itère sur les candidats gardés (`r` de 1 à `kmax`).
 - **Génération du Point de Départ** : Pour chaque itération `r`, un nouveau point de départ `y_samp` est créé :
 - Il est une combinaison linéaire du candidat Sobol actuel (`candidat_garde[r]`) et du meilleur optimum local trouvé dans les itérations précédentes (`x_sample`, obtenu via `max_compare` sur `list_max`).

- Le poids de `x_sample` est contrôlé par λ , qui évolue de 0.25 à 1 au fur et à mesure que `r` augmente. La formule est : $\lambda = 0.25 + 0.75 * (r - 1) / \max(1, k_{\max} - 1)$.
 - `y_samp = (1 -  $\lambda$ ) * candidat_garde[r] +  $\lambda$  * x_sample`
 - Stratégie d'Interpolation : Lorsque `r` augmente, `y_samp` augmente (de 0.25 vers 1).
Par conséquent, le poids du candidat Sobol ($1 - \lambda$) diminue, tandis que le poids de l'optimum local augmente.
 - **Recherche Locale Itérative** : Une recherche locale (fonction dans le code "recherche_local") est effectuée à partir de ce `y_samp`. Le résultat est ajouté à `list_max`.
 - Ainsi, `list_max` accumule une série de points optimisés localement, chacun généré à partir d'un point de départ qui est une interpolation guidée entre les candidats Sobol initiaux et les meilleures solutions locales trouvées jusqu'à présent.
4. **Sélection du Meilleur Global et Initialisation** : La meilleure solution parmi tous les optima locaux obtenus par `cherche` est choisie comme `best_x`. Les composantes importantes de `best_x` sont utilisées pour initialiser le solveur MadNLP final (`x0[ind_x1] .= x1, x0[ind_x2] .= x2`).

Voici les deux importantes fonctions qui résume le Code

```
function cherche(nlp, candidat_garde)

    list_max = Vector{Vector{Float64}}()
    list_new_vect = Vector{Vector{Float64}}()

    y_samp = candidat_garde[1]

    push!(list_new_vect, candidat_garde[1])
    push!(list_max, candidat_garde[1])

    # evolution de lambda
    kmax = length(candidat_garde)

    for r in 1:kmax
        x_sample = max_compare(nlp, list_max)

        if r == 1
            else
                # evolution de lambda
                lambda = 0.25 + 0.75 * (r - 1) / max(1, kmax - 1)
                y_samp = (1 - lambda) .* candidat_garde[r] .+ lambda .* x_sample
            end

            push!(list_new_vect, y_samp)
            push!(list_max, recherche_local(nlp, y_samp))
        end

    end

    return list_max
end

function tiktak(mpcc)

    solver = build_madnlp(mpcc)
    nlp = solver.cb.nlp
    # Get initial primal position
    x0 = NLPModels.get_x0(nlp)

    # Get indices
    ind_x1 = mpcc.meta.ind_cc1
    ind_x2 = mpcc.meta.ind_cc2
    # tiktak
```

```

N = 1000
N_garde = 100

candidat = generate_sobol_candidates(length(x0), N)
candidat_trie = f_obj(nlp, candidat)

candidat_garde = candidat_trie[1:min(N_garde, end)]

best_x = max_compare(nlp, cherche(nlp, candidat_garde))

# TODO: modify initial point here!!!(tiktak)

x1 = best_x[ind_x1]
x2 = best_x[ind_x2]
x0[ind_x1] .= x1
x0[ind_x2] .= x2

solver = MadNLPSolver(
nlp;
print_level = MadNLP.ERROR
)

results = MadNLP.solve!(solver)

# Get complementarity solution
x1_sol = results.solution[ind_x1]
x2_sol = results.solution[ind_x2]

println(length(results.solution))

return results
end

```

Cet Algorithme bien que simple reste intéressant mais ne présente pas des résultats assez concluant quant à l'optimalité des points trouvés dans le cas de nos géants modèles. On essaiera d'échantillonner un peu mieux sur l'espace en passant d'un échantillonnage de Sobol à du Monte-Carlo Hamiltonian pour espérer de meilleurs résultats, tout en gardant la structure du TikTak classique.